

Computer Evolution

❖ “Computer evolution and Technology used in each step”.

- Electromechanical Computers (1930s - 1940s):
 - ✓ Technology: Relays and switches.
 - ✓ Examples: Z3 by Konrad Zuse, Harvard Mark I.
- Vacuum Tube Computers (1940s - 1950s):
 - ✓ Technology: Vacuum tubes for electronic circuitry.
 - ✓ Examples: ENIAC, UNIVAC I.
- Transistorized Computers (1950s - 1960s):
 - ✓ Technology: Transistors, smaller, more reliable than vacuum tubes.
 - ✓ Examples: IBM 7090, DEC PDP-1.
- Integrated Circuit Computers (1960s - 1970s):
 - ✓ Technology: Integrated circuits (ICs) containing multiple transistors on a single chip.
 - ✓ Examples: IBM System/360, DEC PDP-11.
- Personal Computers and Workstations (1970s - Present):
 - ✓ Technology: Graphical User Interfaces (GUI), operating systems (e.g., Windows, macOS, Linux), storage devices (HDDs, SSDs).
 - ✓ Examples: IBM PC, Apple Macintosh, various PC clones.
- Networking and Internet Age (1980s - Present):
 - ✓ Technology: Ethernet, TCP/IP protocol suite, modems, broadband internet, Wi-Fi.
 - ✓ Examples: ARPANET (precursor to the internet), World Wide Web, social media platforms.
- Cloud Computing and Artificial Intelligence (2010s - Present):
 - ✓ Technology: Cloud services (AWS, Azure, Google Cloud), machine learning algorithms, neural networks.
 - ✓ Examples: Amazon Web Services (AWS), Google Cloud Platform, AI-driven applications

Virtual Memory

1. Briefly describe the virtual memory in the computer system.

Virtual memory is like a backup storage area that your computer uses when it runs out of space in its main memory (RAM).

Let's imagine you're playing a video game on your computer that requires a lot of memory. Your computer's RAM is like a small desk, and it can only hold a certain amount of information at once. As you play the game, you might have multiple levels, characters, and details loaded into the RAM.

Now, let's say the game is really big, and your computer's RAM is not large enough to hold everything at the same time. This is where virtual memory comes in. Think of virtual memory as an extension of your desk, like a shelf or a drawer. When the game needs more space than the RAM can provide, it stores some less frequently used information on this virtual memory space.

2. Give the capabilities of virtual memory.

- I. It uses main memory efficiently by treating it as a cache for an address space stored on disk, keeping only the active areas in main memory, and transferring data back and forth between disk and memory as needed.
- II. It simplifies memory management by providing each process with a uniform address space.
- III. It protects the address space of each process from corruption by other processes.

3. Explain page hits and page faults in virtual memory.

Page Hits: Consider what happens when the CPU reads a word of virtual memory contained in VP 2, which is cached in DRAM (Figure 9.5). The address translation hardware uses the virtual address as an index to locate PTE 2 and read it from memory. Since the valid bit is set, the address translation hardware knows that VP 2 is cached in memory. So it uses the physical memory address in the PTE (which points to the start of the cached page in PP 1) to construct the physical address of the word.

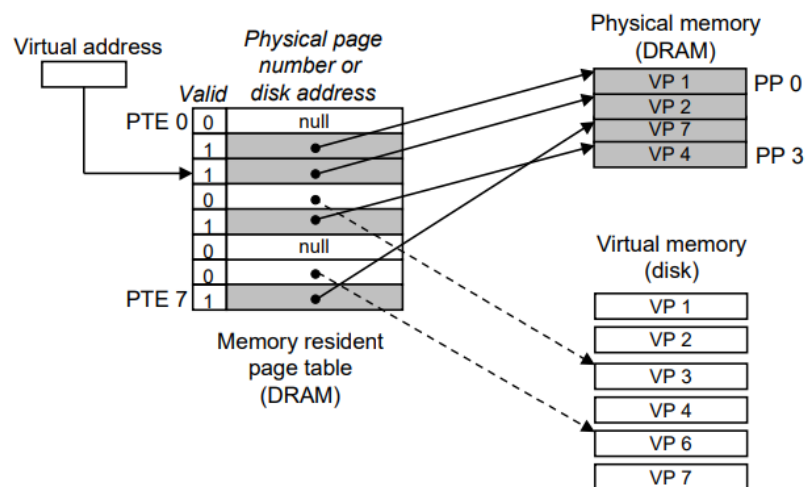


Figure 9.5: **VM page hit.** The reference to a word in VP 2 is a hit.

Page Faults: In virtual memory parlance, a DRAM cache miss is known as a page fault. Figure 9.6 shows the state of our example page table before the fault. The CPU has referenced a word in VP 3, which is not cached in DRAM. The address translation hardware reads PTE 3 from memory, infers from the valid bit that VP 3 is not cached, and triggers a page fault exception.

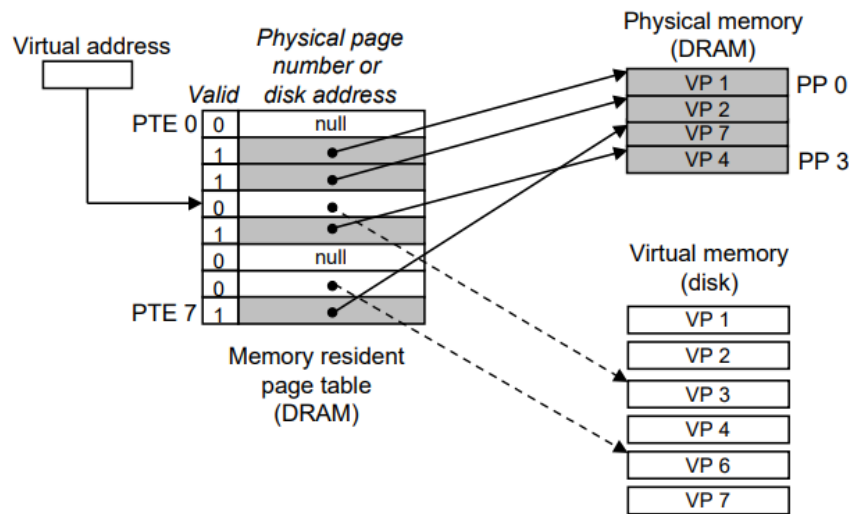


Figure 9.6: **VM page fault (before).** The reference to a word in VP 3 is a miss and triggers a page fault.

The page fault exception invokes a page fault exception handler in the kernel, which selects a victim page, in this case VP 4 stored in PP 3. If VP 4 has been modified, then the kernel copies it back to disk. In either case, the kernel modifies the page table entry for VP 4 to reflect the fact that VP 4 is no longer cached in main memory.

Next, the kernel copies VP 3 from disk to PP 3 in memory, updates PTE 3, and then returns. When the handler returns, it restarts the faulting instruction, which resends the faulting virtual address to the address translation hardware. But now, VP 3 is cached in main memory, and the page hit is handled normally by the address translation hardware. Figure 9.7 shows the state of our example page table after the page fault.

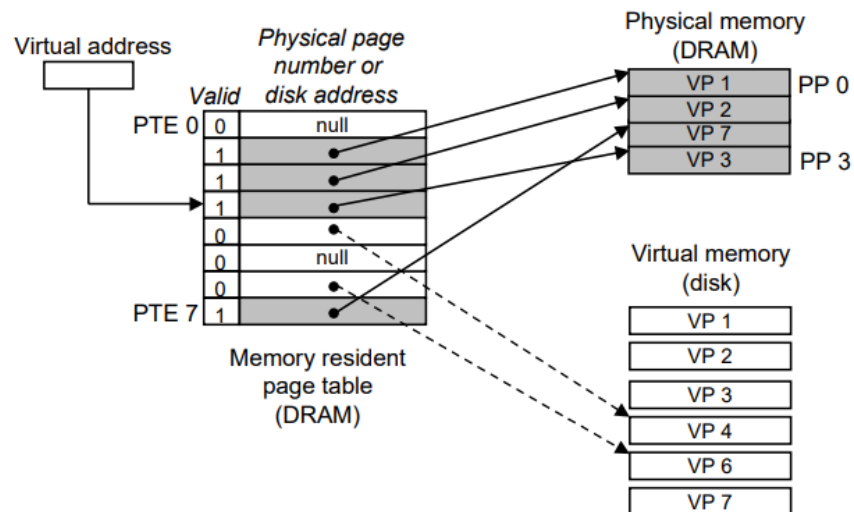


Figure 9.7: **VM page fault (after).** The page fault handler selects VP 4 as the victim and replaces it with a copy of VP 3 from disk. After the page fault handler restarts the faulting instruction, it will read the word from memory normally, without generating an exception.

4. Determine the number of page table entries that are needed for the following combinations of virtual address size (n) and page size (P). (Solution)

n	$P = 2^p$	#PTE
16	4K	16
64	16K	2^{50}

The number of page table entries (PTE) needed can be calculated using the formula:

Number of PTE = Virtual Address Size/Page Size

Let's calculate the number of PTE for the given combinations:

Problem-1: First row of table,

Given virtual address size 2^{16}

Given page size is 4K = 4×1024 bytes = 2^{12} bytes.

Therefore, the number of page table entries (PTEs) = $2^{16}/2^{12} = 2^4 = 16$ entries.

Problem-2: Second row of table,

Given virtual address size 2^{64}

Given page size is 16K = 16×1024 bytes = 2^{14} bytes.

Therefore, the number of Page Table Entries (PTEs) = $2^{64}/2^{14} = 2^{50}$ entries

n	$P = 2^p$	#PTE
16	2K	
32	4K	
64	8K	

Same as above

Less Important topics of VM but must read.

5. How Physical and Virtual Address works?

The main memory of a computer system is organized as an array of M contiguous byte-sized cells. Each byte has a unique physical address (PA). The first byte has an address of 0, the next byte an address of 1, the next byte an address of 2, and so on. Given this simple organization, the most natural way for a CPU to access memory would be to use physical addresses. We call this approach physical addressing. Figure 9.1 shows an example of physical addressing in the context of a load instruction that reads the word starting at physical address 4.

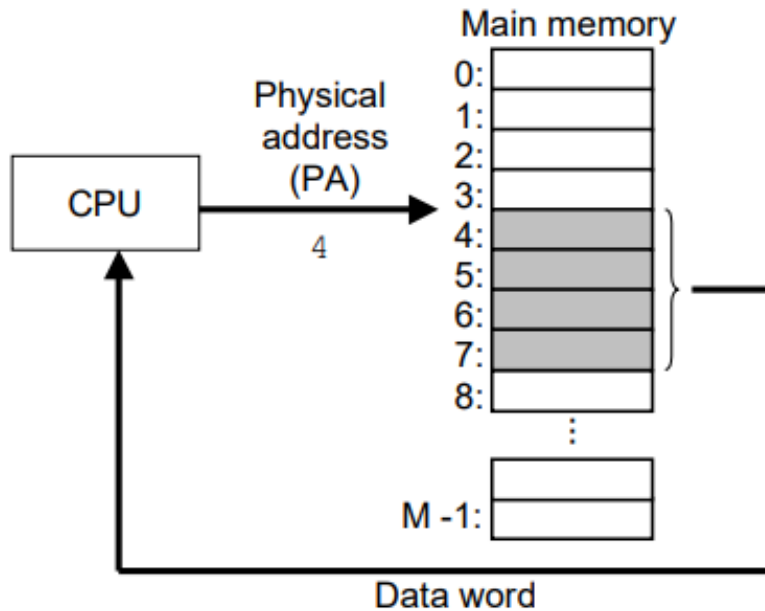


Figure 9.1: A system that uses physical addressing.

When the CPU executes the load instruction, it generates an effective physical address and passes it to main memory over the memory bus. The main memory fetches the 4-byte word starting at physical address 4 and returns it to the CPU, which stores it in a register.

However, modern processors use a form of addressing known as virtual addressing, as shown in Figure 9.2.

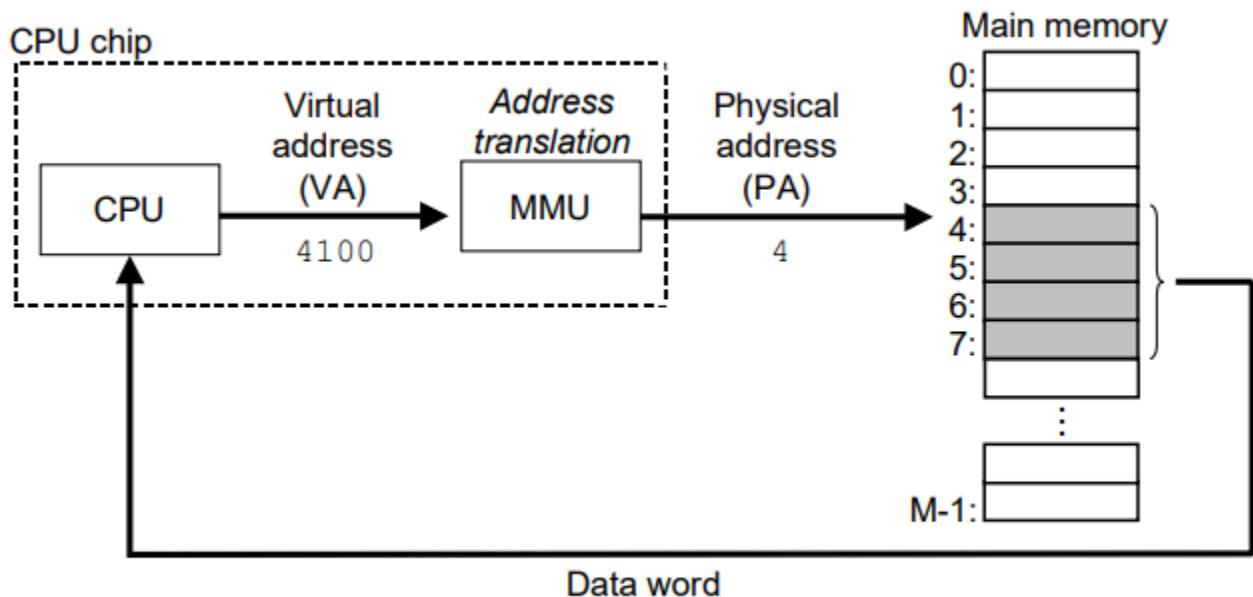


Figure 9.2: A system that uses virtual addressing.

With virtual addressing, the CPU accesses main memory by generating a virtual address (VA), which is converted to the appropriate physical address before being sent to the memory. The task of converting a virtual address to a physical one is known as address translation. Like exception handling, address translation requires close cooperation between the CPU hardware and the operating system. Dedicated hardware on the CPU chip called the memory management unit (MMU) translates virtual addresses on the fly, using a look-up table stored in main memory whose contents are managed by the operating system

6. VM as a Tool for Caching.

Conceptually, a virtual memory is organized as an array of N contiguous byte-sized cells stored on disk. Each byte has a unique virtual address that serves as an index into the array. The contents of the array on disk are cached in main memory. As with any other cache in the memory hierarchy, the data on disk (the lower level) is partitioned into blocks that serve as the transfer units between the disk and the main memory (the upper level). VM systems handle this by partitioning the virtual memory into fixed-sized blocks called virtual pages (VPs). Each virtual page is $P = 2^p$ bytes in size. Similarly, physical memory is partitioned into physical pages (PPs), also P bytes in size. (Physical pages are also referred to as page frames.)

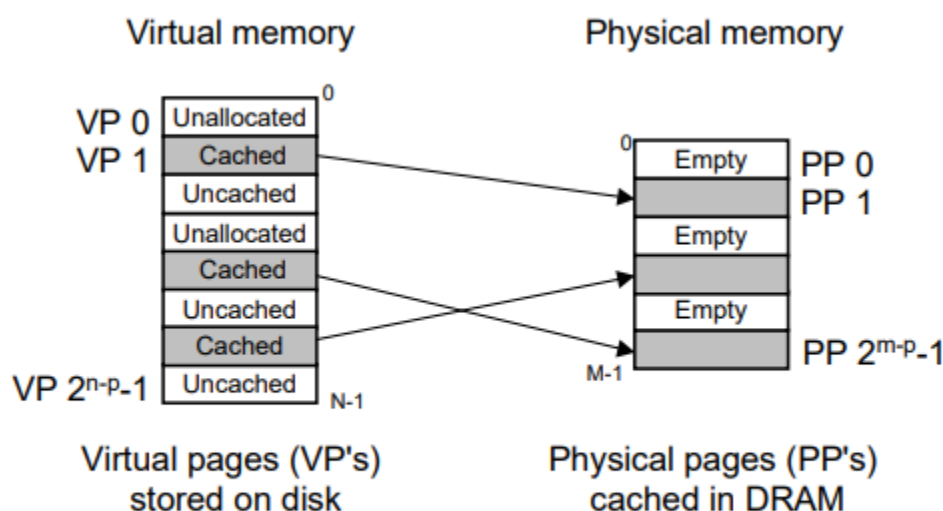


Figure 9.3: How a VM system uses main memory as a cache.

At any point in time, the set of virtual pages is partitioned into three disjoint subsets:

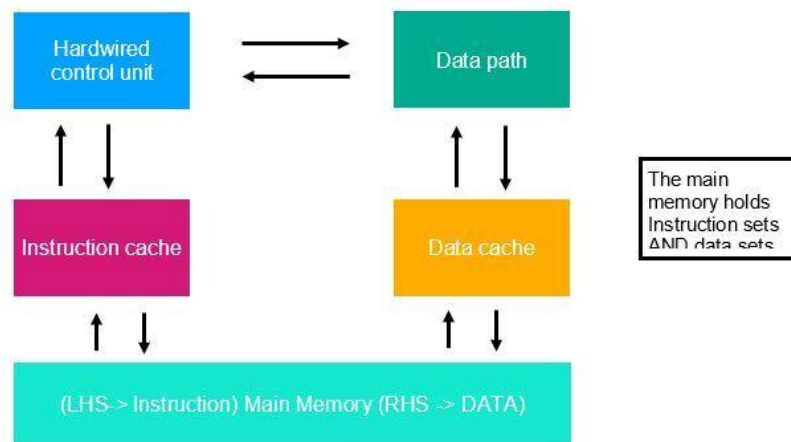
- **Unallocated:** Pages that have not yet been allocated (or created) by the VM system. Unallocated blocks do not have any data associated with them, and thus do not occupy any space on disk.
- **Cached:** Allocated pages that are currently cached in physical memory.
- **Uncached:** Allocated pages that are not cached in physical memory.

RISC & CISC

1. Give the characteristics of RISC and CISC.

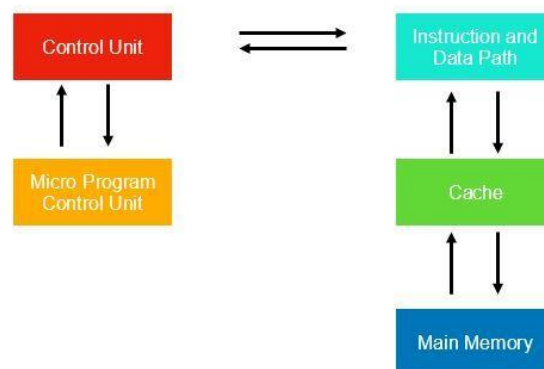
➤ RISC (Reduced Instruction Set Computer) Characteristics:

1. It has simpler instructions and thus simple instruction decoding.
2. More general-purpose registers.
3. The instruction takes one clock cycle in order to get executed.
4. The instruction comes under the size of a single word.
5. Pipeline can be easily achieved.
6. Few data types.
7. Simpler addressing modes.



➤ CISC (Complex instruction Set Computer) Characteristics:

1. Instructions are complex, and thus it has complex instruction decoding.
2. The instructions may take more than one clock cycle in order to get executed.
3. The instruction is larger than one-word size.
4. Lesser general-purpose registers since the operations get performed only in the memory.
5. More data types.
6. Complex addressing modes.



2. RISC vs CISC

RISC	CISC
Focus on software	Focus on hardware
It has simpler instructions and thus simple instruction decoding.	Instructions are complex, and thus it has complex instruction decoding.
More general-purpose registers	Lesser general-purpose registers since the operations get performed only in the memory.
The instruction takes one clock cycle in order to get executed	The instruction is larger than one-word size.
Few data types.	More data types.
Simpler addressing modes	Complex addressing modes.

Performance Measure

The time taken by the CPU to execute a job can be expressed as We denote the number of CPU clock cycles for executing a job to be the cycle

count (CC), the cycle time by CT, and the clock frequency by $f = 1/CT$. The time taken by the CPU to execute a job can be expressed as:

$$\text{CPU time} = \text{CC} \times \text{CT} = \text{CC}/f$$

In the case that the CPI for each instruction category is known, the overall CPI can be computed as

$$CPI = \frac{\sum_{i=1}^n CPI_i \times I_i}{\text{Instruction count}}$$

where I_i is the number of times an instruction of type i is executed in the program and CPI_i is the average number of clock cycles needed to execute such instruction.

Example Consider computing the overall CPI for a machine A for which the following performance measures were recorded when executing a set of benchmark programs. Assume that the clock rate of the CPU is 200 MHz.

Instruction category	Percentage of occurrence	No. of cycles per instruction
ALU	38	1
Load & store	15	3
Branch	42	4
Others	5	5

Assuming the execution of 100 instructions, the overall CPI can be computed as

$$CPI_a = \frac{\sum_{i=1}^n CPI_i \times I_i}{\text{Instruction count}} = \frac{38 \times 1 + 15 \times 3 + 42 \times 4 + 5 \times 5}{100} = 2.76$$

It should be noted that the CPI reflects the organization and the instruction set architecture of the processor while the instruction count reflects the instruction set architecture and compiler technology used. This shows the degree of interdependence between the two performance parameters. Therefore, it is imperative that both the

- ⊛ A different performance measure that has been given a lot of attention in recent years is MIPS (million instructions-per-second (the rate of instruction execution per unit time)), which is defined as:

$$MIPS = \frac{\text{Instruction count}}{\text{Execution time} \times 10^6} = \frac{\text{Clock rate}}{CPI \times 10^6}$$

Example Suppose that the same set of benchmark programs considered above were executed on another machine, call it machine B, for which the following measures were recorded.

Instruction category	Percentage of occurrence	No. of cycles per instruction
ALU	35	1
Load & store	30	2
Branch	15	3
Others	20	5

What is the *MIPS* rating for the machine considered in the previous example (machine A) and machine B assuming a clock rate of 200 MHz?

$$CPI_a = \frac{\sum_{i=1}^n CPI_i \times I_i}{\text{Instruction count}} = \frac{38 \times 1 + 15 \times 3 + 42 \times 4 + 5 \times 5}{100} = 2.76$$

$$MIPS_a = \frac{\text{Clock rate}}{CPI_a \times 10^6} = \frac{200 \times 10^6}{2.76 \times 10^6} = 70.24$$

$$CPI_b = \frac{\sum_{i=1}^n CPI_i \times I_i}{\text{Instruction count}} = \frac{35 \times 1 + 30 \times 2 + 20 \times 5 + 15 \times 3}{100} = 2.4$$

$$MIPS_b = \frac{\text{Clock rate}}{CPI_b \times 10^6} = \frac{200 \times 10^6}{2.4 \times 10^6} = 83.67$$

Thus $MIPS_b > MIPS_a$.

It is interesting to note here that although MIPS has been used as a performance measure for machines, one has to be careful in using it to compare machines having different instruction sets. This is because

MIPS does not track execution time. Consider, for example, the following measurement made on two different machines running a given set of benchmark programs.

Instruction category	No. of instructions (in millions)	No. of cycles per instruction
Machine (A)		
ALU	8	1
Load & store	4	3
Branch	2	4
Others	4	3
Machine (B)		
ALU	10	1
Load & store	8	2
Branch	2	4
Others	4	3

$$CPI_a = \frac{\sum_{i=1}^n CPI_i \times I_i}{\text{Instruction count}} = \frac{(8 \times 1 + 4 \times 3 + 4 \times 3 + 2 \times 4) \times 10^6}{(8 + 4 + 4 + 2) \times 10^6} \cong 2.2$$

$$MIPS_a = \frac{\text{Clock rate}}{CPI_a \times 10^6} = \frac{200 \times 10^6}{2.2 \times 10^6} \cong 90.9$$

$$CPU_a = \frac{\text{Instruction count} \times CPI_a}{\text{Clock rate}} = \frac{18 \times 10^6 \times 2.2}{200 \times 10^6} = 0.198 \text{ s}$$

$$CPI_b = \frac{\sum_{i=1}^n CPI_i \times I_i}{\text{Instruction count}} = \frac{(10 \times 1 + 8 \times 2 + 4 \times 4 + 2 \times 4) \times 10^6}{(10 + 8 + 4 + 2) \times 10^6} = 2.1$$

$$MIPS_b = \frac{\text{Clock rate}}{CPI_b \times 10^6} = \frac{200 \times 10^6}{2.1 \times 10^6} = 95.2$$

$$CPU_b = \frac{\text{Instruction count} \times CPI_b}{\text{Clock rate}} = \frac{20 \times 10^6 \times 2.1}{200 \times 10^6} = 0.21 \text{ s}$$

$$MIPS_b > MIPS_a \quad \text{and} \quad CPU_b > CPU_a$$

Micro-Operations

Let us consider an example for clear understanding of Micro-Operations:

Consider an ADD instruction:

ADD R1, X

Which adds the contents of the location X to register R1. The following sequence of micro-operations might occur:

$t_1: \text{MAR} \leftarrow \{\text{IR}:(\text{address})\}$

$t_2: \text{MBR} \leftarrow \text{Memory}$

$t_3: \text{R1} \leftarrow \{\text{R1}\} + \{\text{MBR}\}$

we begin with the IR containing the ADD instruction.

- ✓ In the first step, the address portion of the IR is loaded into the MAR.
- ✓ Then the referenced memory location is read.
- ✓ Finally, the contents of R1 and MBR are added by the ALU.